

McCulloch–Pitts Neurons, Perceptron, ADALINE, and the delta rule

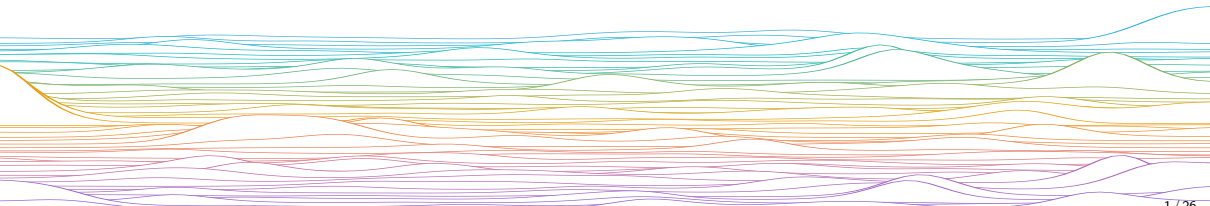
Computational Neuroscience
University of Bristol

M Rule

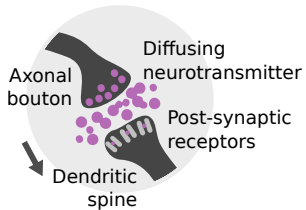


Learning outcomes:

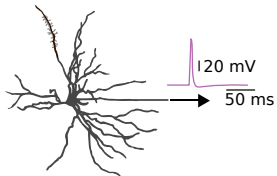
- ▶ Describe the general features of linear–nonlinear artificial “neurons” (units)
 - ...and how these may (or may not) relate to biological neurons and biological learning
- ▶ Definition and intuition behind the delta-rule



Biological

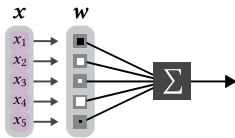
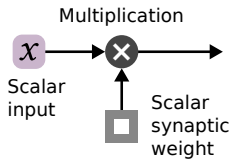


Dendritic integration

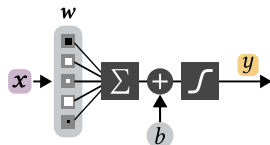


Firing threshold
+ Action potential

Artificial



Linear summation

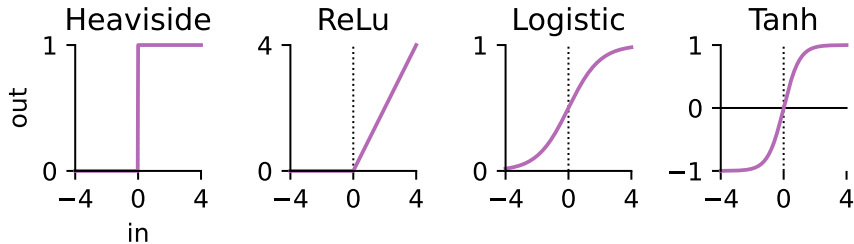


Additive bias
+ Saturating nonlinearity

Where is the computation?

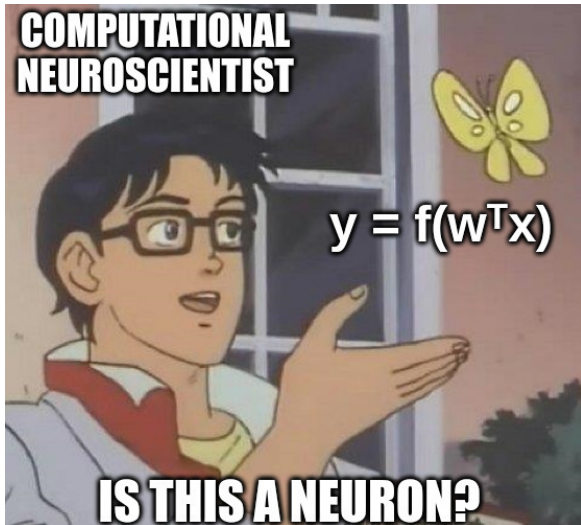
$$y = \phi(\mathbf{w}^T \mathbf{x}) = \phi(\sum_i w_i x_i) \quad (1)$$

For **activation function** $\phi(\cdot)$, consider



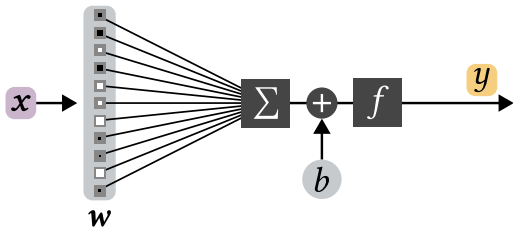
(many others)

**COMPUTATIONAL
NEUROSCIENTIST**



$$y = f(w^T x)$$

IS THIS A NEURON?



$$\begin{aligned}
 y(x; \mathbf{w}, b) &= f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b) \\
 &= f(\sum_{i=1\dots n} w_ix_i + b) \\
 &= f(\mathbf{w}^\top \mathbf{x} + b)
 \end{aligned}$$

$$y \in \mathbb{R}$$

$$\mathbf{x} = \{x_1, \dots, x_n\}^\top \in \mathbb{R}^n$$

$$\mathbf{w} = \{w_1, \dots, w_n\}^\top \in \mathbb{R}^n$$

$$b \in \mathbb{R}$$

output

inputs

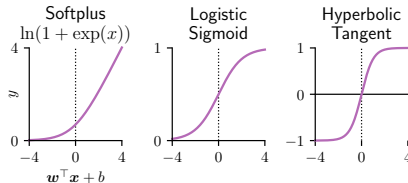
weights

bias

Artificial neuron (unit):

- 1 Take weighted sum of inputs $\mathbf{w}^\top \mathbf{x} = \sum_i w_ix_i$
- 2 Add an offset $+b$
- 3 Apply a nonlinear activation function $f(\cdot)$.

$f(\cdot)$ is a nonlinear activation function e.g.



What is the simplest “unit” that could arguably be a model of a neuron?

McCulloch-Pitts “Neuron”

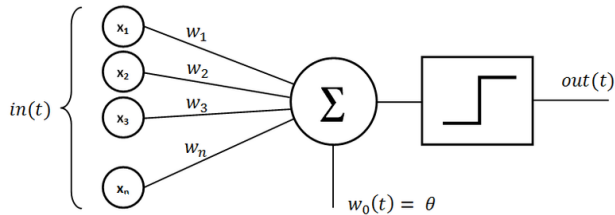
Warren McCulloch, Walter Pitts (1943)

Weighted linear combination + binary threshold

$$y = H(\mathbf{w}^\top \mathbf{x}) \in \{0, 1\}$$

Heaviside step

$$H(a) = \begin{cases} 1 & a > 0 \\ 0 & \text{otherwise} \end{cases}$$



The Perceptron

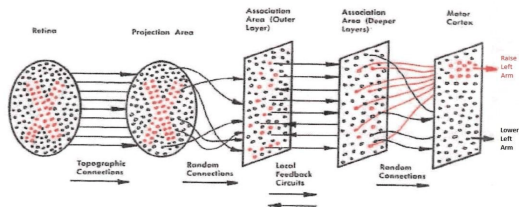


FIG. 1 — Organization of a biological brain. (Red areas indicate active cells, responding to the letter X.)

Frank Rosenblatt (1958) Designed for black-and-white image classification

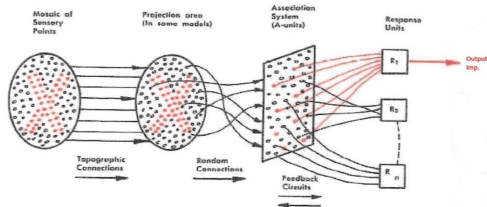


FIG. 2 — Organization of a perceptron.

[Rosenblatt \(1958\)](#)

ADALINE (*Adaptive Linear Neuron*)

Bernard Widrow, Marcian Hoff (1960)
For adaptive signal processing

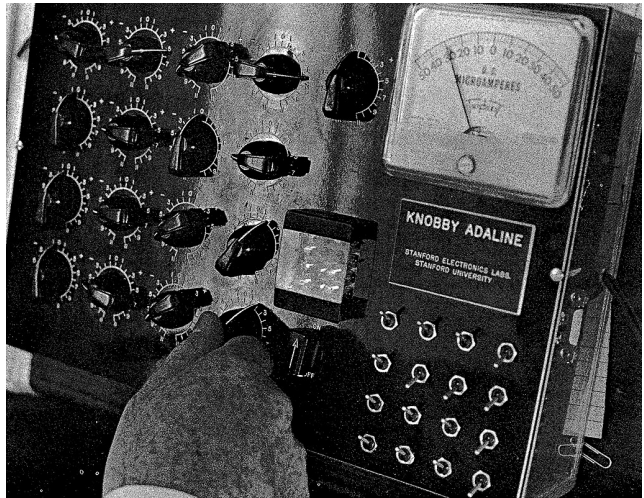


Photo of an ADALINE machine, via [wikimedia](#)

The “delta” (Widrow Hoff) learning rule

Historically, this is called the Delta “ δ ” rule, but...

For notational consistency I will be using $\varepsilon = \text{target} - \text{prediction}$ for “errors” rather than δ .

Delta (Widrow-Hoff) Rule

Let

- ▶ Inputs $\mathbf{x} = \{x_1, \dots, x_n\}^\top$
- ▶ Weights $\mathbf{w} = \{w_1, \dots, w_n\}^\top$
- ▶ Output (prediction) y
- ▶ Target y^*
- ▶ Learning rate η

Computation:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} = \sum_i w_i x_i$$

Error ε

$$\varepsilon = \underline{y^* - y}$$

Weight update:

$$\underline{\Delta w_i = \eta \varepsilon x_i}$$

$$\Delta w \propto (y^* - y) x_i$$

$$\Delta w \propto y^* x_i$$

Delta rule: intuition

$$\Delta \mathbf{w}^\top = \eta \underbrace{(y^* - \overbrace{\mathbf{w}^\top \mathbf{x}}^y)}_{\varepsilon} \mathbf{x}^\top \quad (2)$$

Assuming inputs x_i are positive:

- ▶ If prediction is too low, $y < y^*$ then $\varepsilon > 0$ and $\varepsilon x_i > 0$, and Δw_i will increase w_i .
- ▶ Next time, $(w_i + \Delta w_i)x_i$ will lead to a larger y .

Only update the weights for active inputs.

- ▶ $x_i = 0, \Rightarrow \Delta w_i = 0$.

The delta rule: Example

Let:

- ▶ Input $\mathbf{x}^T = \{0, 1\}$
- ▶ Initial weights $\mathbf{w}^T = \{1, 1\}$
- ▶ Target $y^* = 3$
- ▶ learning rate $\eta = 0.1$

$$\begin{array}{cc} x & w \\ 0 & \cdot 1 = 0 \\ 1 & \cdot 1 = 1 \end{array} \quad \Delta \quad \Sigma \rightarrow 1$$
$$y^* = 3 - 1 = 2$$

Calculate:

- ▶ Output $y = \mathbf{w}^T \mathbf{x} = 1$
- ▶ Error $\varepsilon = y^* - \hat{y} = 3 - 1 = 2$

$$0.2 \cdot \eta = 0 \cdot 2 = 0$$

$$1 \cdot 2 \cdot \eta = 2 \cdot \eta = 0.2$$

Update weights:

$$\Delta w_1 = \eta \varepsilon x_1 = 0.1 \cdot 2 \cdot 0 = 0$$

$$\Delta w_2 = \eta \varepsilon x_2 = 0.1 \cdot 2 \cdot 1 = 0.2$$

New weights: $\mathbf{w}^T = \{1, 1.2\}$

Next time $y = w_1 \cdot x_1 + w_2 \cdot x_2 = 1.2$ is closer to the target.

$$x_1 \cdot w_2 = 1.2$$

closer to 3 than 1

The delta rule is just online gradient descent in a linear neuron

Motivation via gradient descent (1/3)

Define a loss function: the squared error between the target and output

$$\mathcal{L} = \frac{1}{2}(y^* - y)^2 = \frac{1}{2}(y^* - \sum_i w_i x_i)^2.$$

Motivation via gradient descent (2/3)

By the definition of the loss,

$$\frac{\partial \mathcal{L}}{\partial w_k} = \frac{1}{2} \frac{\partial}{\partial w_k} \underbrace{\left(y^* - \sum_i w_i x_i \right)}_{\varepsilon}^2$$

Applying the chain rule,

$$\frac{\partial \mathcal{L}}{\partial w_k} = (y^* - \sum_i w_i x_i) \frac{\partial}{\partial w_k} (y^* - \sum_i w_i x_i)$$

Noting that the gradient of y , x_i and $w_{k \neq i}$ are all zero,

$$\frac{\partial \mathcal{L}}{\partial w_k} = (y^* - \sum_i w_i x_i) \frac{\partial}{\partial w_k} (-w_i x_i)$$

$$\frac{\partial \mathcal{L}}{\partial w_k} = (y^* - \sum_i w_i x_i) (-x_k)$$

$$\frac{\partial \mathcal{L}}{\partial w_k} = -\varepsilon x_k$$

Motivation via gradient descent (3/3)

To go downhill a bit, we need:

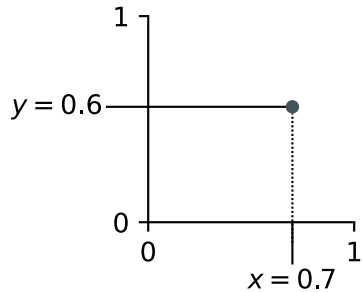
$$\Delta w_i = -\eta \frac{\partial \mathcal{L}}{\partial w_k}$$

(the gradient points uphill, so we use minus the gradient to go downhill).

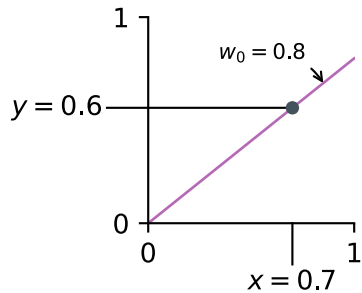
Substituting the gradient from the previous slide, we get back the delta rule,

$$\Delta w_i = \eta \varepsilon x_i$$

$$\hat{y} = w x$$

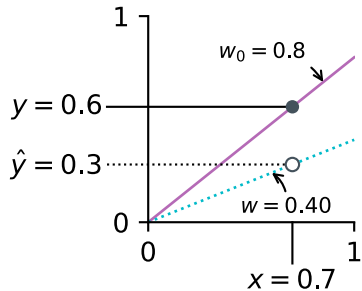


$$\hat{y} = w x$$



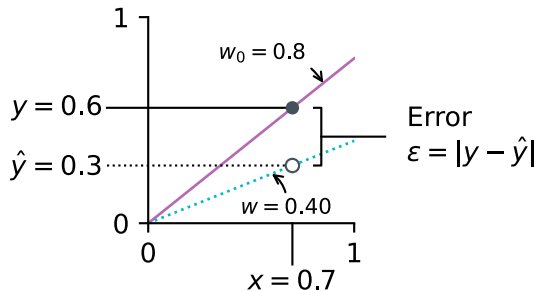
— True $y = w_0 x$

$$\hat{y} = w x$$



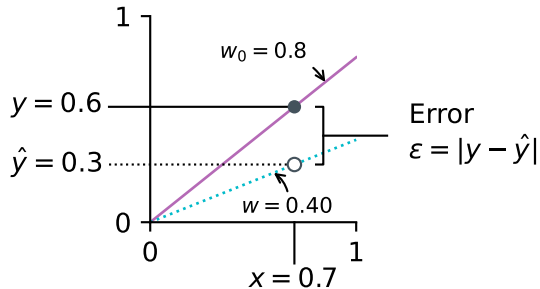
— True $y = w_0 x$ Model $\hat{y} = w x$

$$\hat{y} = w x$$

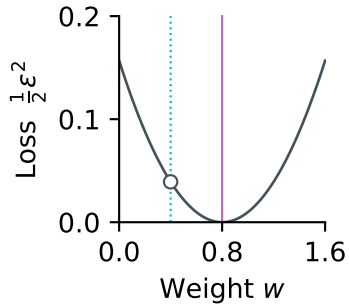


— True $y = w_0 x$ Model $\hat{y} = w x$

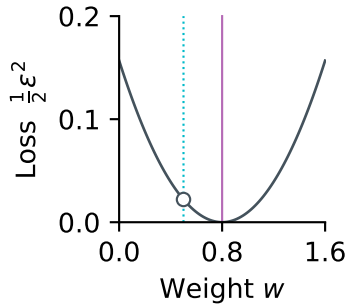
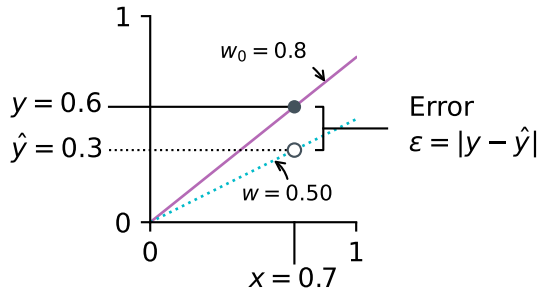
$$\hat{y} = w x$$



— True $y = w_0 x$ Model $\hat{y} = w x$

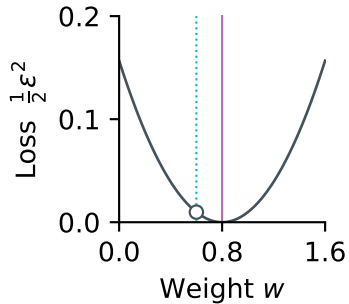
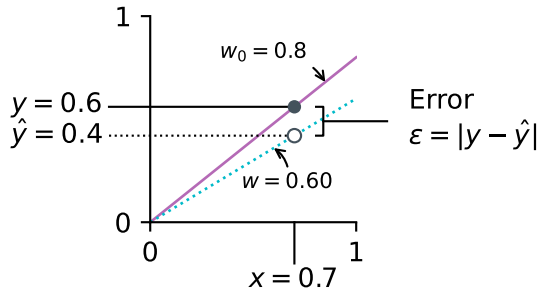


$$\hat{y} = w x$$



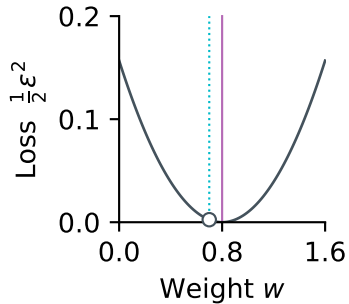
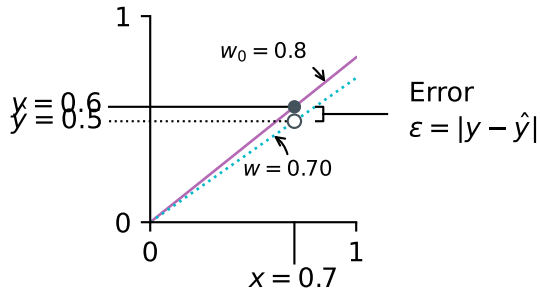
— True $y = w_0 x$ Model $\hat{y} = w x$

$$\hat{y} = w x$$



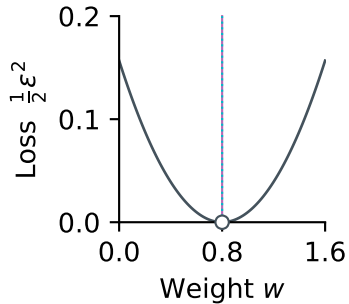
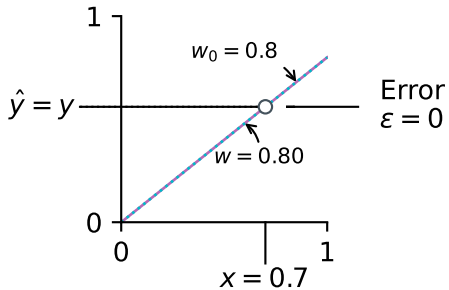
— True $y = w_0 x$ Model $\hat{y} = w x$

$$\hat{y} = w x$$



— True $y = w_0 x$ Model $\hat{y} = w x$

$$\hat{y} = wx - b$$

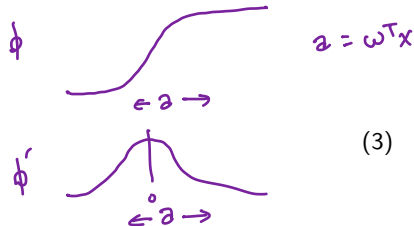


— True $y = w_0 x$ Model $\hat{y} = wx$

Gradient descent, more generally

The original Delta (Widrow-Hoff) rule is for a linear unit

$$\Delta \mathbf{w}^\top = \eta \underbrace{(y^* - \mathbf{w}^\top \mathbf{x})}_\varepsilon \mathbf{x}^\top$$



(3)

For generic loss $\mathcal{L}(y^*, y)$ and more general linear-nonlinear unit $y(\mathbf{x}; \mathbf{w}) = \phi(a)$ (where “ $a = \mathbf{w}^\top \mathbf{x}$ ” is the synaptic activation), you might see

$$\begin{aligned} \Delta w_i &= -\eta \cdot \frac{d\mathcal{L}}{dy} \cdot \frac{dy}{da} \cdot \frac{da}{dw_i} \\ &= \eta \cdot \underbrace{[-\partial_y \mathcal{L}]}_\varepsilon \cdot \phi'(a) \cdot x_i \end{aligned}$$

Realism note:

- ▶ The brain does not literally perform gradient descent. But, it is generally assumed that activity-driven synaptic changes are structured to (approximately) minimise *some* loss function (even if we're not quite sure what that loss function is). The extent to which this assumption is correct remains to be seen.

Least Mean Squares (LMS)



Legal disclaimer:

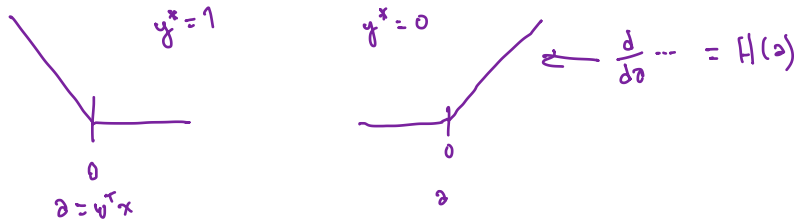
- Formally, the “delta rule” is equivalent to online gradient descent of a *linear* “neuron” (AKA least-mean-squares). In this course we will also use the term “delta rule” to refer to the supervised learning algorithm for a perceptron classifier. The mathematical forms of these rules are identical, differing only in the nature of the supervised training signal.

Common question:

- In the “perceptron” lab, why is it OK to use a learning rule designed for a linear neuron with a quadratic loss? The Heaviside step function (binary threshold) in the McCulloch-Pitts “Neuron” isn’t even differentiable?

Simple answer

- The updates still generally point in the right direction, the only problem is possible overshooting near the threshold. If you use a small learning rate, you can’t overshoot that much.



Self-test

For a linear-nonlinear unit $y = \phi(a)$ with $a = \mathbf{w}^\top \mathbf{x}$, state the $\approx$ biological interpretations of

- ▶ Output y : _____, or firing rate (post-synaptic cell) spikes
- ▶ Nonlinearity ϕ : Spiking threshold or firing-rate nonlinearity
- ▶ Activation a : membrane voltage depolarization at the soma
- ▶ Weights $\mathbf{w}$: maximal postsynaptic conductance e.g. AMPAR density in glutamate synapse
- ▶ Inputs $\mathbf{x}$: Spikes (presynaptic) (always positive for real neurons)

In a McCulloch-Pitts neuron $y = H(\mathbf{w}^\top \mathbf{x})$

- ▶ H is a binary threshold / Heaviside firing-rate nonlinearity

In a delta-style learning rule $\Delta \mathbf{w} \propto (y^* - \mathbf{w}^\top \mathbf{x})\mathbf{x}$ for a linear neuron

- ▶ y^* is a target
- ▶ $y^* - \mathbf{w}^\top \mathbf{x}$ can be interpreted as a error
 $\underbrace{\hspace{1cm}}_{\Sigma}$

Self-test

For a linear-nonlinear unit $y = \phi(a)$ with $a = \mathbf{w}^\top \mathbf{x}$, state the $\approx$ biological interpretations of

- ▶ Output y : spikes, or firing rate
- ▶ Nonlinearity ϕ : spiking threshold, or firing-rate nonlinearity
- ▶ Activation a : soma/dendrite depolarization / ~~open~~ conductances
- ▶ Weights $\mathbf{w}$: synaptic weights e.g. AMPAR density in glutamate synapse
- ▶ Inputs $\mathbf{x}$: spiking inputs (always positive for real neurons)

In a McCulloch-Pitts neuron $y = H(\mathbf{w}^\top \mathbf{x})$

- ▶ H is a binary threshold / Heaviside function

In a delta-style learning rule $\Delta \mathbf{w} \propto (y^* - \mathbf{w}^\top \mathbf{x})\mathbf{x}$ for a linear neuron

- ▶ y^* is a supervised target
- ▶ $y^* - \mathbf{w}^\top \mathbf{x}$ can be interpreted as a prediction error

end

